

Python Language Programming

Object Oriented Programming (OOP)

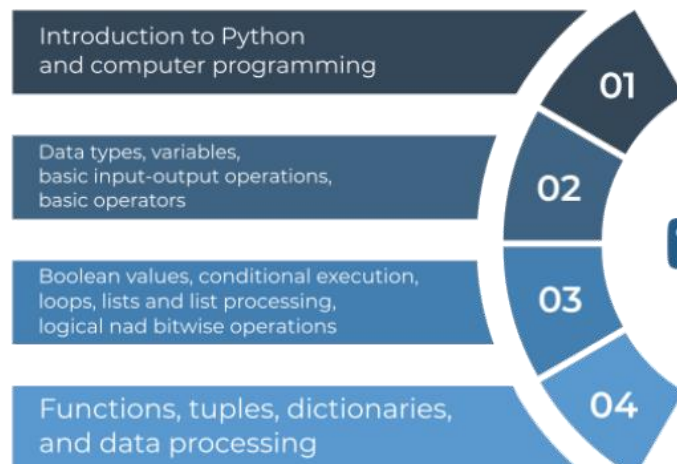
- **Disclaimer**

- These slides and notes are based on the contents of the courses:

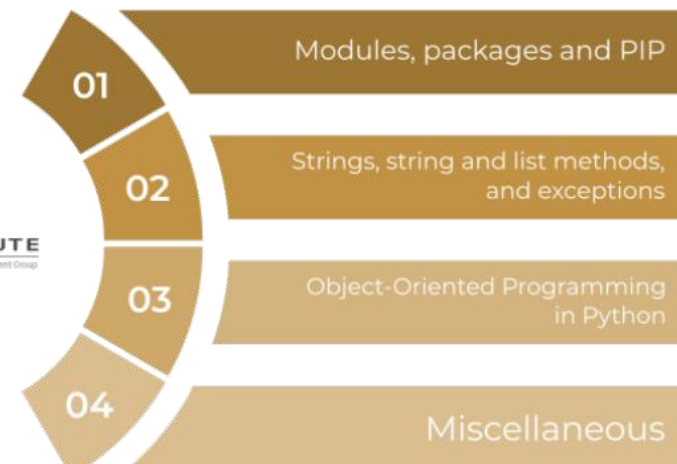
- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

Python Essentials 1



Python Essentials 2



- **Disclaimer**

- These slides and notes are based on the contents of the courses:

- **Python Essentials 1 and 2**

- aligned with the **PCEP** and **PCAP** certifications, respectively;
- developed by the OpenEDG Python Institute;
(www.pythoninstitute.org)
- offered in the Cisco Networking Academy program.

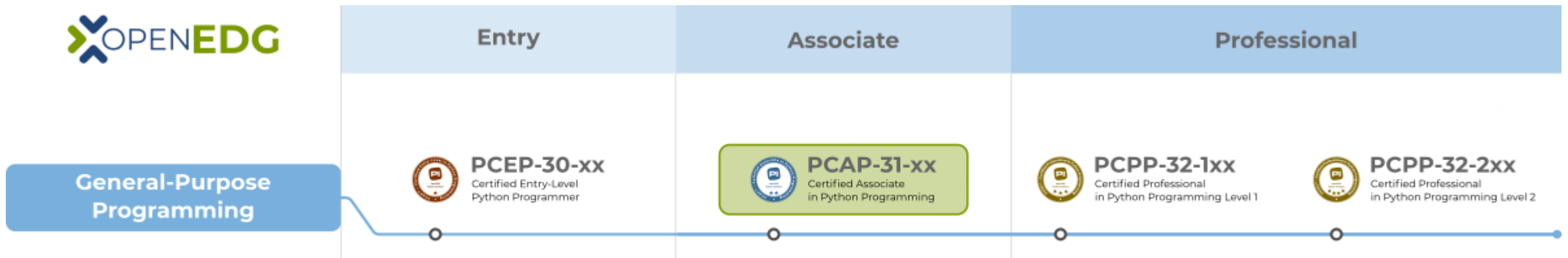


Table of Contents

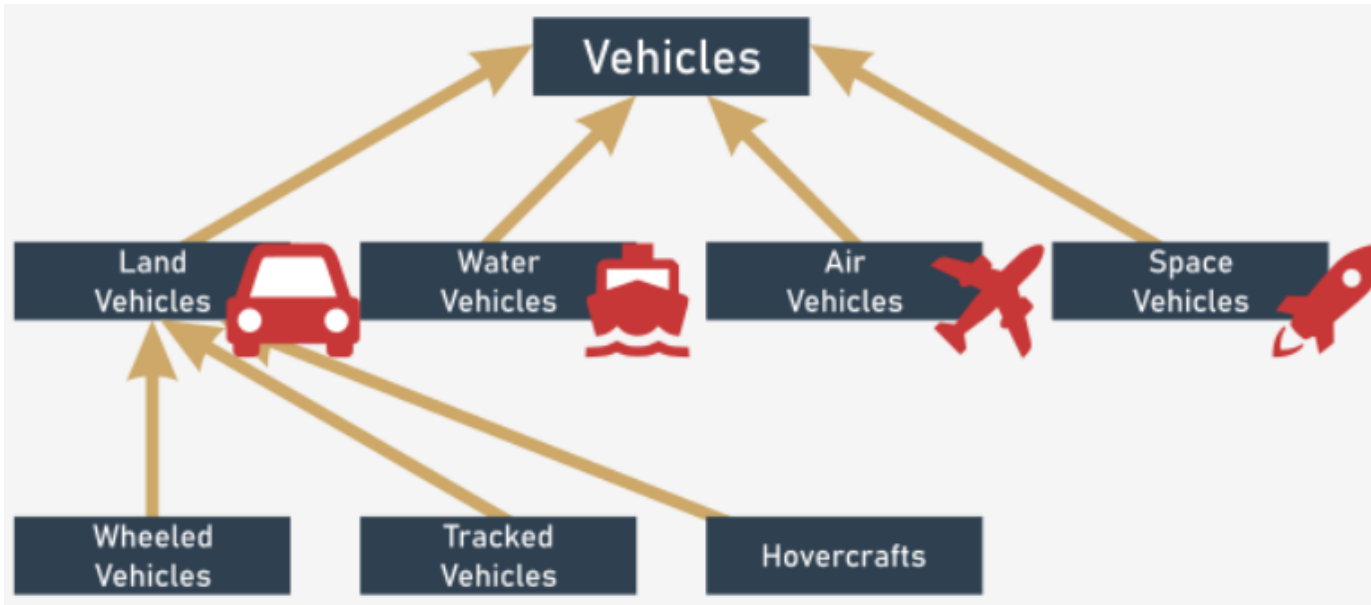
- **Definition**
- **Classes**
- **Objects**
- **Variables**
 - Instance variables
 - Class variables
- **Checking an attribute's existence**
 - through exceptions
 - through `hasattr()` function
- **Methods**
 - `self` parameter
 - `__init__` method (constructor)
 - `__str__` method
- **Inheritance**
 - Definition
 - Object's entity location
 - Overriding
 - Multiple inheritance
- **Functions and Operators**
- **Bibliography**

Python Language Programming

- Object Oriented Programming (OOP)
 - Definition
 - a **class**:
 - like a recipe which can be used to create a useful object
 - an **object**:
 - has a set of traits/variables (called properties)
 - able to perform a set of activities (called methods).
 - The objects interact with each other, exchanging data or activating their methods.

Python Language Programming

- Object Oriented Programming (OOP)
 - Classes
 - class is like a category, as a result of precisely defined similarities



The Vehicles class is very broad. We have to define some more specialized classes. The specialized classes are the **subclasses**. The vehicles class will be a **superclass** for them all.

```
class TheSimplestClass:  
    pass
```

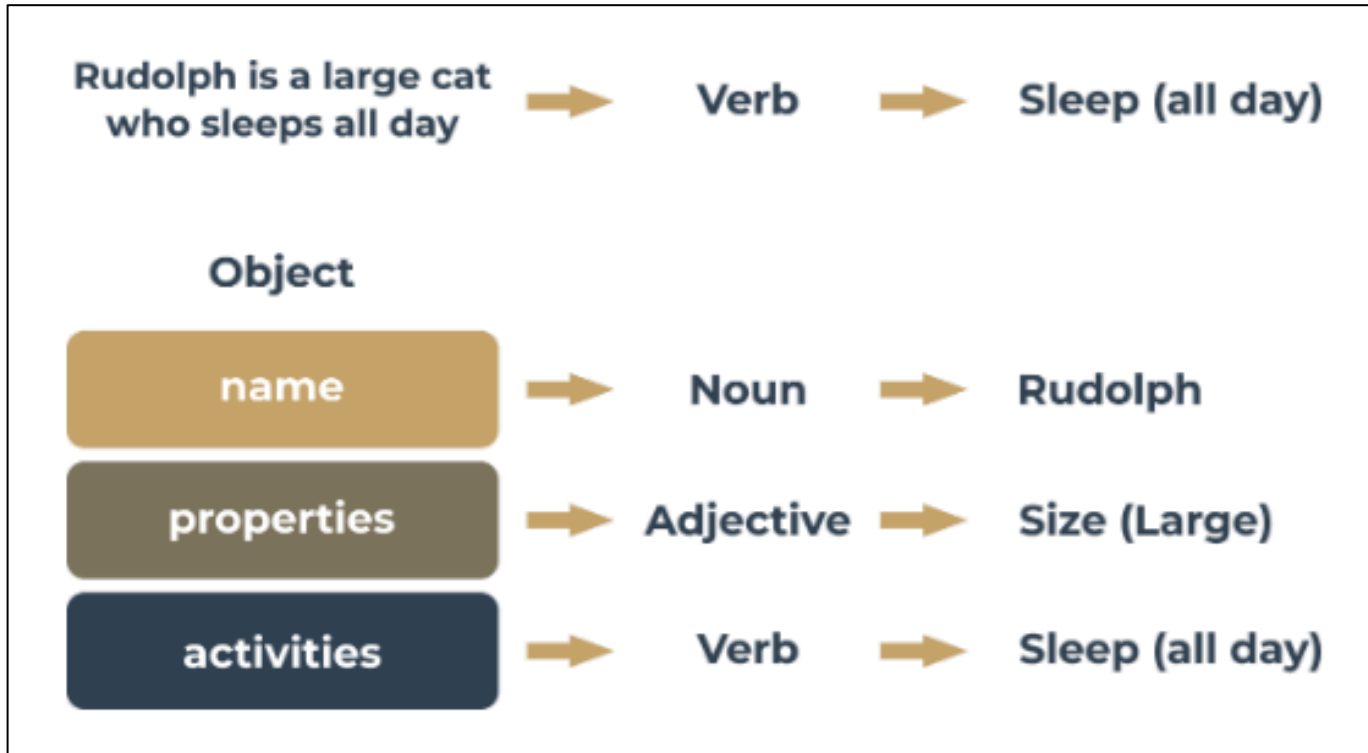
pass keyword fills the class with nothing. It doesn't contain any methods or properties

Python Language Programming

- **Object Oriented Programming (OOP)**
 - **Objects**
 - an object is an incarnation of the traits and activities assigned to a specific class
 - every existing object may be equipped with three groups of attributes:
 - **name**: that uniquely identifies it within its home namespace
 - **traits/properties (variables)**: set of traits which make it original, unique, or outstanding
 - **activities/methods (functions)**: change the object itself, or some of the other objects.

Python Language Programming

- Object Oriented Programming (OOP)
 - Objects

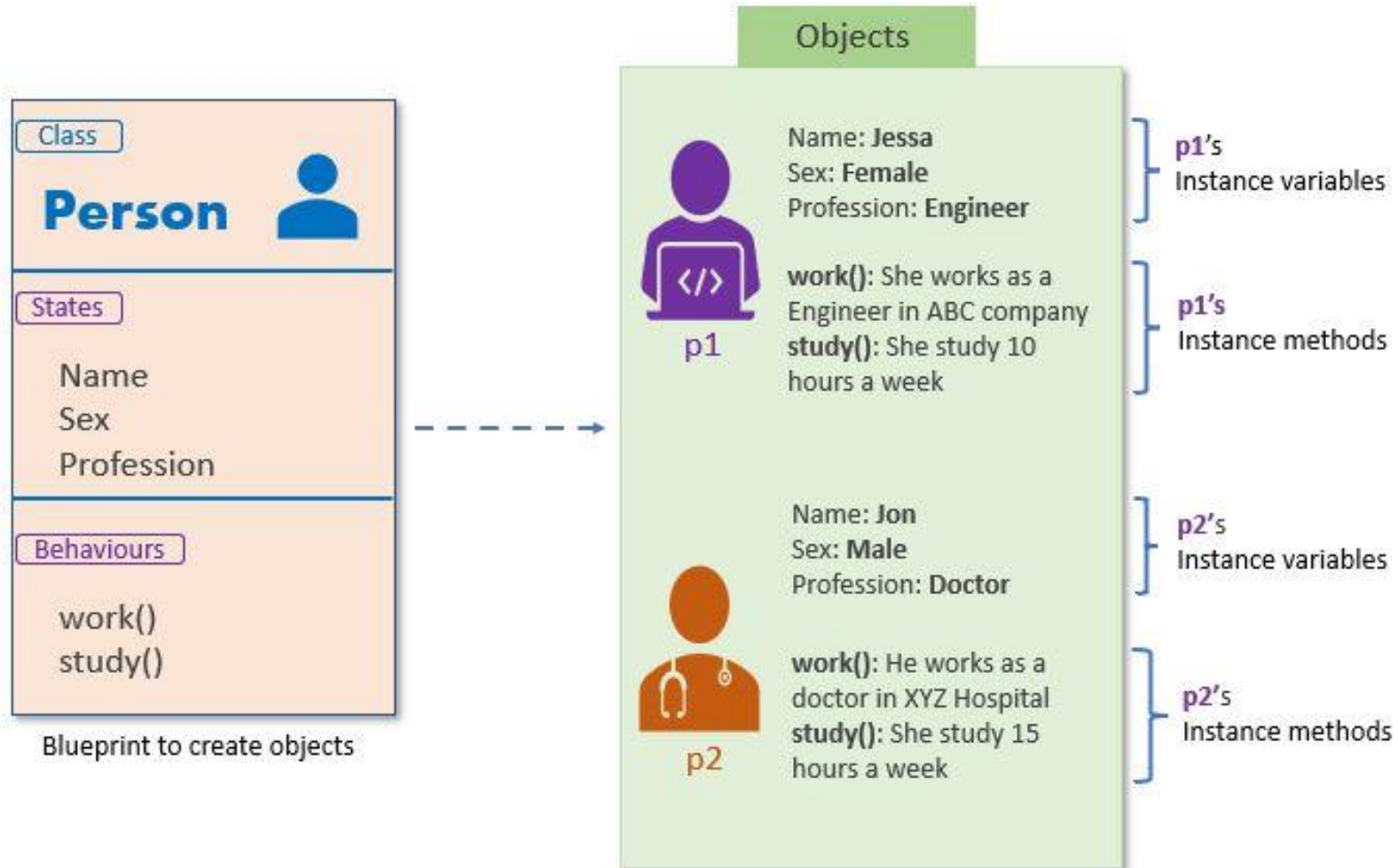


How to create an object: treat the class name like a function, returning a newly instantiated object of the class.

```
my_first_object = TheSimplestClass()
```


Python Language Programming

- Object Oriented Programming (OOP)
 - Objects



Python Language Programming

• Object Oriented Programming (OOP)

```
class Stack:
    def __init__(self):
        self.__stack_list = []

    def push(self, val):
        self.__stack_list.append(val)

    def pop(self):
        val = self.__stack_list[-1]
        del self.__stack_list[-1]
        return val
```

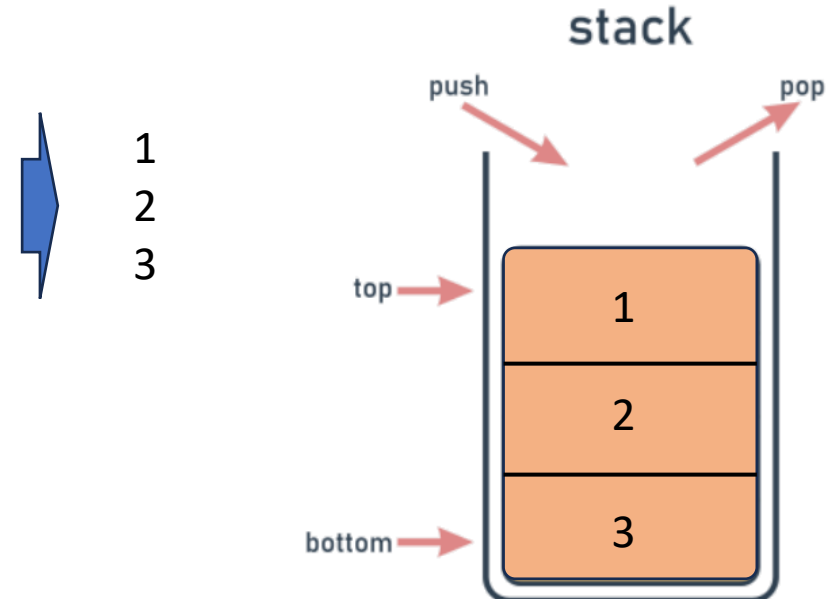
```
stack_object = Stack()
```

```
stack_object.push(3)
stack_object.push(2)
stack_object.push(1)
print(stack_object.pop())
print(stack_object.pop())
print(stack_object.pop())
```

constructor **__init__**

- the mandatory parameter named **self** at the first position of the parameters list - it's only a convention but should be followed. It allows a method to access entities (properties and activities/methods) carried out by the actual object

any class component with a name starting with two underscores (**__**) becomes private - **encapsulation**



Python Language Programming

- Object Oriented Programming (OOP)
 - Instance variables
 - different objects of the same class may possess different sets of instance variables;

```
class ExampleClass:
    def __init__(self, val = 1):
        self.first = val
    def set_second(self, val):
        self.second = val

example_object_1 = ExampleClass()
example_object_2 = ExampleClass(2)
example_object_2.set_second(3)
example_object_3 = ExampleClass(4)
example_object_3.third = 5
print(example_object_1.__dict__)
print(example_object_2.__dict__)
print(example_object_3.__dict__)
```



{'first': 1}
{'second': 3, 'first': 2}
{'third': 5, 'first': 4}

__dict__ (it's a dictionary): the variable contains the names and values of all the properties (variables) the object (or class) is currently carrying

Python Language Programming

- Object Oriented Programming (OOP)
 - Class variables
 - a property which exists in just one copy and is stored outside any object;

```
class ExampleClass:
    counter = 0
    def __init__(self, val = 1):
        self.first = val
        ExampleClass.counter += 1
example_object_1 = ExampleClass()
example_object_2 = ExampleClass(2)
example_object_3 = ExampleClass(4)
print(example_object_1.__dict__, example_object_1.counter)
print(example_object_2.__dict__, example_object_2.counter)
print(example_object_3.__dict__, example_object_3.counter)
```



```
{'first': 1} 3
{'first': 2} 3
{'first': 4} 3
```

Python Language Programming

- Object Oriented Programming (OOP)
 - Checking an attribute's existence
 - through exceptions
 - in contrast to other programming languages, you may NOT expect that all objects of the same class have the same sets of properties.
 - accessing a non-existing object (class) attribute causes an **AttributeError** exception.

```
class ExampleClass:
    def __init__(self, val):
        if val % 2 != 0:
            self.a = 1
        else:
            self.b = 2

example_object = ExampleClass(5)
print(example_object.a)
print(example_object.b)
```



```
1
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 11, in
<module>
AttributeError: 'ExampleClass' object
has no attribute 'b'
```

Python Language Programming

- Object Oriented Programming (OOP)
 - Checking an attribute's existence
 - through exceptions
 - in contrast to other programming languages, you may NOT expect that all objects of the same class have the same sets of properties.
 - accessing a non-existing object (class) attribute causes an `AttributeError` exception.

```
class ExampleClass:
    def __init__(self, val):
        if val % 2 != 0:
            self.a = 1
        else:
            self.b = 2

example_object = ExampleClass(5)
print(example_object.a)

try:
    print(example_object.b)
except AttributeError:
    pass
```



Python Language Programming

- Object Oriented Programming (OOP)
 - Checking an attribute's existence
 - through `hasattr()` function
 - checks if any object/class contains a specified property.
 - expects two arguments:
 - the class or the object being checked;
 - the name (string) of the property to check
 - returns True or False

```
class ExampleClass:
    a = 1
    def __init__(self):
        self.b = 2

example_object = ExampleClass()

print(hasattr(example_object, 'b'))
print(hasattr(example_object, 'a'))
print(hasattr(ExampleClass, 'b'))
print(hasattr(ExampleClass, 'a'))
```



True
True
False
True

Python Language Programming

- Object Oriented Programming (OOP)
 - Checking an attribute's existence
 - through `hasattr()` function
 - checks if any object/class contains a specified property.
 - expects two arguments:
 - the class or the object being checked;
 - the name (string) of the property to check
 - returns True or False

```
class ExampleClass:  
    attr = 1  
  
print(hasattr(ExampleClass, 'attr'))  
print(hasattr(ExampleClass, 'prop'))
```



True
False

Python Language Programming

- Object Oriented Programming (OOP)
 - Methods
 - method is a function embedded inside a class
 - it is mandatory to have at least one parameter – it may be invoked without an argument, but not declared without parameters:
 - the first (or only) parameter is usually named self which identifies the object for which the method is invoked.

```
class Classy:  
    def method(self):  
        print("method")
```

```
obj = Classy()  
obj.method()
```



method

Python Language Programming

- Object Oriented Programming (OOP)
 - Methods
 - if a method needs to accept parameters other than `self`:
 - place them **after** `self` in the method's definition;
 - deliver them during invocation without specifying `self`

```
class Classy:  
    def method(self, var):  
        print("method:", var)
```

```
obj = Classy()  
obj.method(1)  
obj.method(2)  
obj.method(3)
```



```
method: 1  
method: 2  
method: 3
```

Python Language Programming

- Object Oriented Programming (OOP)
 - Methods
 - `self` parameter
 - used to obtain access to the object's instance and class variables.

```
class Classy:
    varia = 2
    def method(self):
        print(self.varia, self.var)

obj = Classy()
obj.var = 3
obj.method()
```



2 3

Python Language Programming

- Object Oriented Programming (OOP)
 - Methods
 - `self` parameter
 - also used to invoke other object/class methods from inside the class.

```
class Classy:
    def other(self):
        print("other")
    def method(self):
        print("method")
        self.other()

obj = Classy()
obj.method()
```



method
other

Python Language Programming

- Object Oriented Programming (OOP)
 - Methods
 - `__init__` method (constructor)
 - must have the `self` parameter
 - may have more parameters than just `self`
 - can be used to set up the object, i.e., properly initialize its internal state, create instance variables etc.
 - **CANNOT:**
 - return a value: it is designed to return a newly created object and nothing else;
 - be invoked directly: either from the object or from inside the class (however a constructor can be invoked from any of the object's subclasses)

Python Language Programming

- Object Oriented Programming (OOP)
 - Methods
 - `__init__` method (constructor)

```
class Classy:
    def __init__(self, value):
        self.var = value

obj_1 = Classy("object")

print(obj_1.var)
```



object

```
class Classy:
    def __init__(self, value = None):
        self.var = value

obj_1 = Classy("object")
obj_2 = Classy()

print(obj_1.var)
print(obj_2.var)
```



object
None

Python Language Programming

- Object Oriented Programming (OOP)
 - Methods
 - `__str__` method
 - invoked when an object is put as argument in the `print()` function

```
class Star:
    def __init__(self, var1, var2):
        self.name = var1
        self.galaxy = var2

    def __str__(self):
        return self.name + ' in ' + self.galaxy

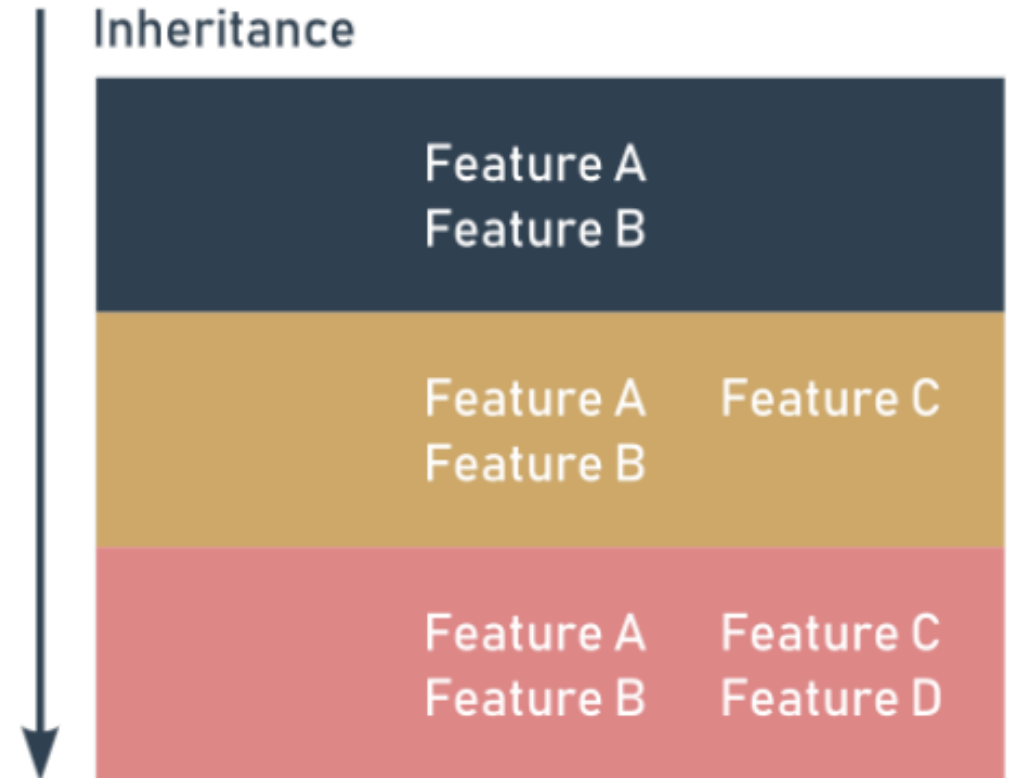
sun = Star("Sun", "Milky Way")
print(sun)
```



Sun in Milky Way

Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Definition
 - a common practice (in object programming) of passing attributes and methods from the superclass (defined and existing) to a newly created class, called the **subclass**.
 - The object's home class may define new traits (as well as requirements and qualities) which will be inherited by any of its subclasses.



Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Definition
 - a common practice (in object programming) of passing attributes and methods from the superclass (defined and existing) to a newly created class, called the **subclass**.
 - The object's home class may define new traits (as well as requirements and qualities) which will be inherited by any of its subclasses.



<https://www.scaler.com/topics/method-overriding-in-python/>

Multi-level Inheritance

```
class Vehicle:  
    pass  
  
class LandVehicle(Vehicle) :  
    pass  
  
class TrackedVehicle(LandVehicle) :  
    pass
```

two-level inheritance

Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Object's entity location
 - when trying to access any object's entity, Python will try to find it (in this order):
 1. inside the object itself;
 2. in its superclasses, from bottom to top;
 3. if there is more than one class on a particular inheritance path, scans them from left to right.
 - If all of the above fail, an exception (`AttributeError`) is raised.

Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance

```
class Level1:
    variable_1 = 100
    def __init__(self):
        self.var_1 = 101
    def fun_1(self):
        return 102

class Level2(Level1):
    variable_2 = 200
    def __init__(self):
        super().__init__()
        self.var_2 = 201
    def fun_2(self):
        return 202

obj = Level2()

print(obj.variable_1, obj.var_1, obj.fun_1())
print(obj.variable_2, obj.var_2, obj.fun_2())
```



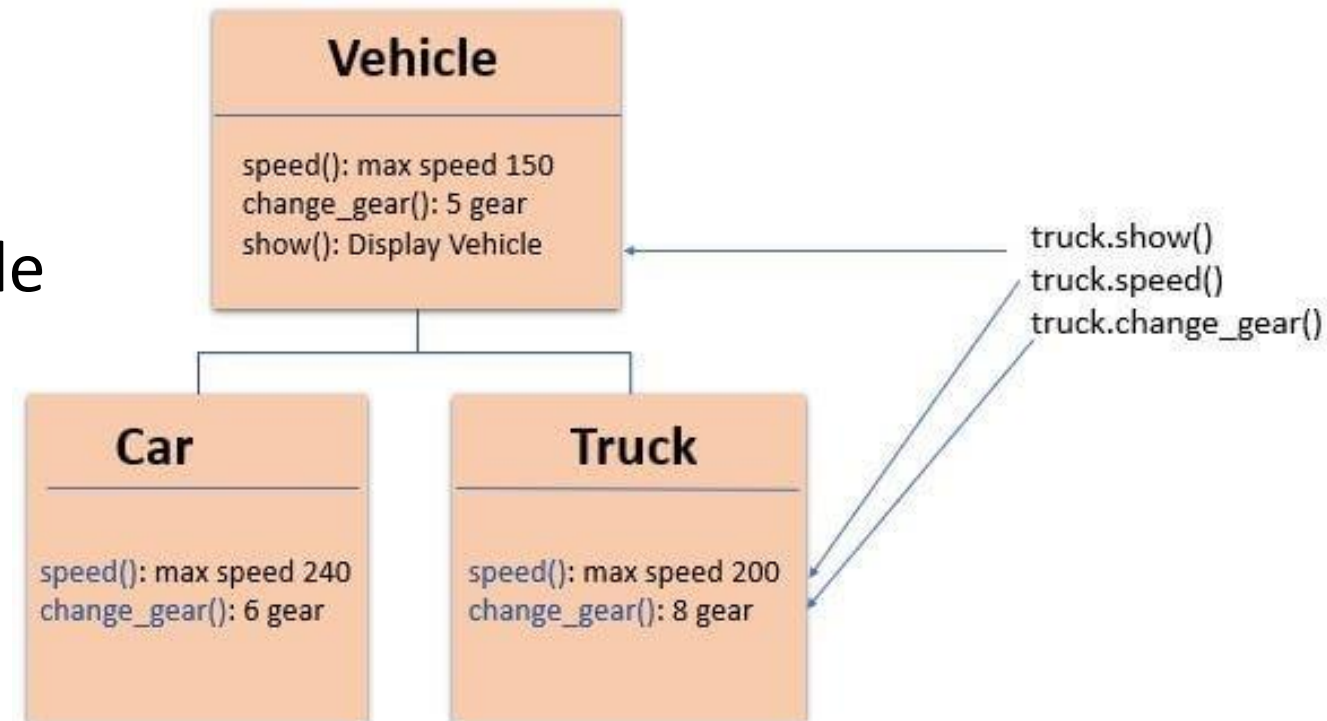
100 101 102
200 201 202

super() function permits to access the superclass without needing to know its name. It returns a reference to the nearest superclass of the class

Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Overriding

- if any of the subclasses defines a method / class variable / instance variable of the same name as existing in the superclass, the new name overrides any of the previous instances of the name.



Method overridden in Car and Truck class

Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Overriding

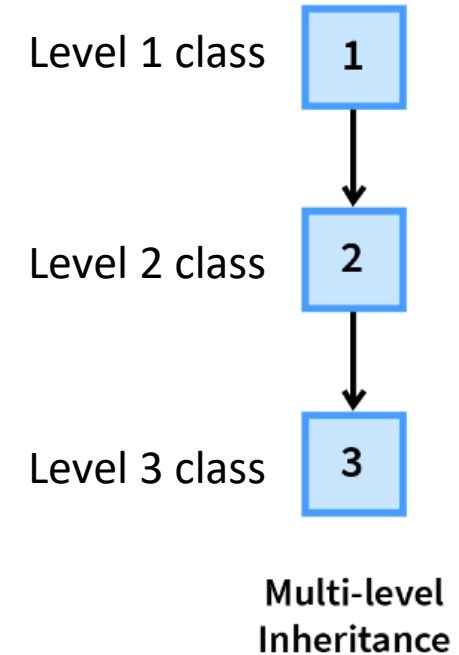
```
class Level1:  
    var = 100  
    def fun(self):  
        return 101  
  
class Level2(Level1):  
    var = 200  
    def fun(self):  
        return 201  
  
class Level3(Level2):  
    pass  
  
obj = Level3()  
  
print(obj.var, obj.fun())
```

Python looks for an entity from
bottom to top



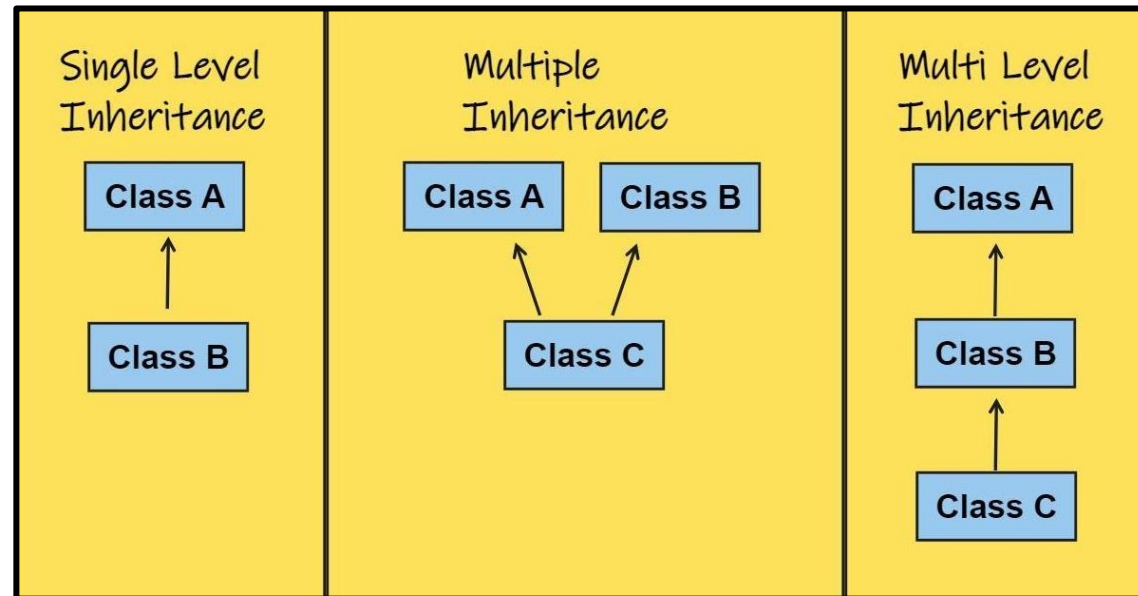
200
201

<https://www.scaler.com/topics/method-overriding-in-python/>



Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Multiple inheritance
 - derive any new class from more than one previously defined classes.



<https://medium.com/@nishi.paul.in/multiple-inheritance-in-python-00ab68fac72d>

Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Multiple inheritance

```
class SuperA:  
    var_a = 10  
    def fun_a(self):  
        return 11
```

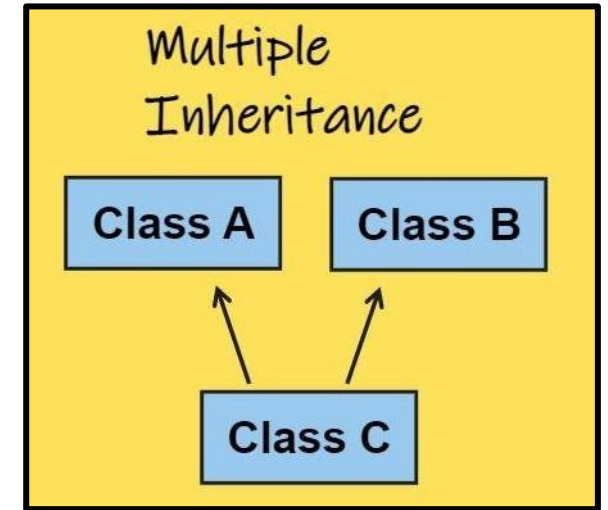
SubC class inherits all the goods offered by both SuperA and SuperB.

```
class SuperB:  
    var_b = 20  
    def fun_b(self):  
        return 21
```

```
class SubC(SuperA, SuperB):  
    pass
```

```
obj = SubC()
```

```
print(obj.var_a, obj.fun_a())  
print(obj.var_b, obj.fun_b())
```



10 11
20 21

Python Language Programming

- Object Oriented Programming (OOP)
 - Inheritance
 - Multiple inheritance

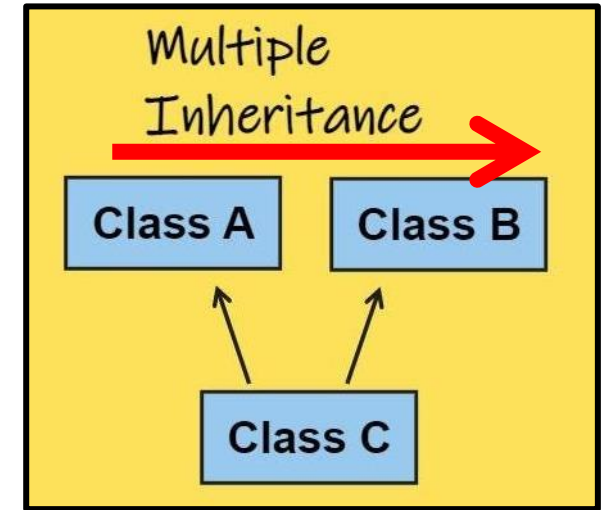
```
class Left:
    var = "L"
    var_left = "LL"
    def fun(self):
        return "Left"
```

```
class Right:
    var = "R"
    var_right = "RR"
    def fun(self):
        return "Right"
```

```
class Sub(Left, Right):
    pass
```

```
obj = Sub()
print(obj.var, obj.var_left, obj.var_right, obj.fun())
```

if there is more than one class
on a particular inheritance path,
**Python scans them from left to
right.**



L LL RR Left

Python Language Programming

- Object Oriented Programming (OOP)
- Example

`__init__` constructor

- the mandatory parameter named **self** at the first position of the parameters list - it's only a convention but should be followed. It allows a method to access entities (properties/variables and activities/methods) carried out by the actual object

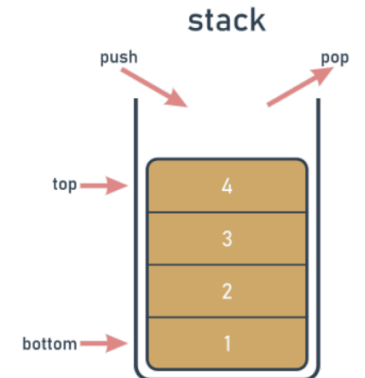
any class component with a name starting with two underscores (`__`) becomes **private - encapsulation**

```
class Stack:
```

```
    def __init__(self):  
        self.__stack_list = []  
  
    def push(self, val):  
        self.__stack_list.append(val)  
  
    def pop(self):  
        val = self.__stack_list[-1]  
        del self.__stack_list[-1]  
        return val
```

```
stack_object = Stack()
```

```
stack_object.push(3)  
stack_object.push(2)  
stack_object.push(1)  
print(stack_object.pop())  
print(stack_object.pop())  
print(stack_object.pop())
```



Python Language Programming

- Object Oriented Programming (OOP)
 - Example (cont)

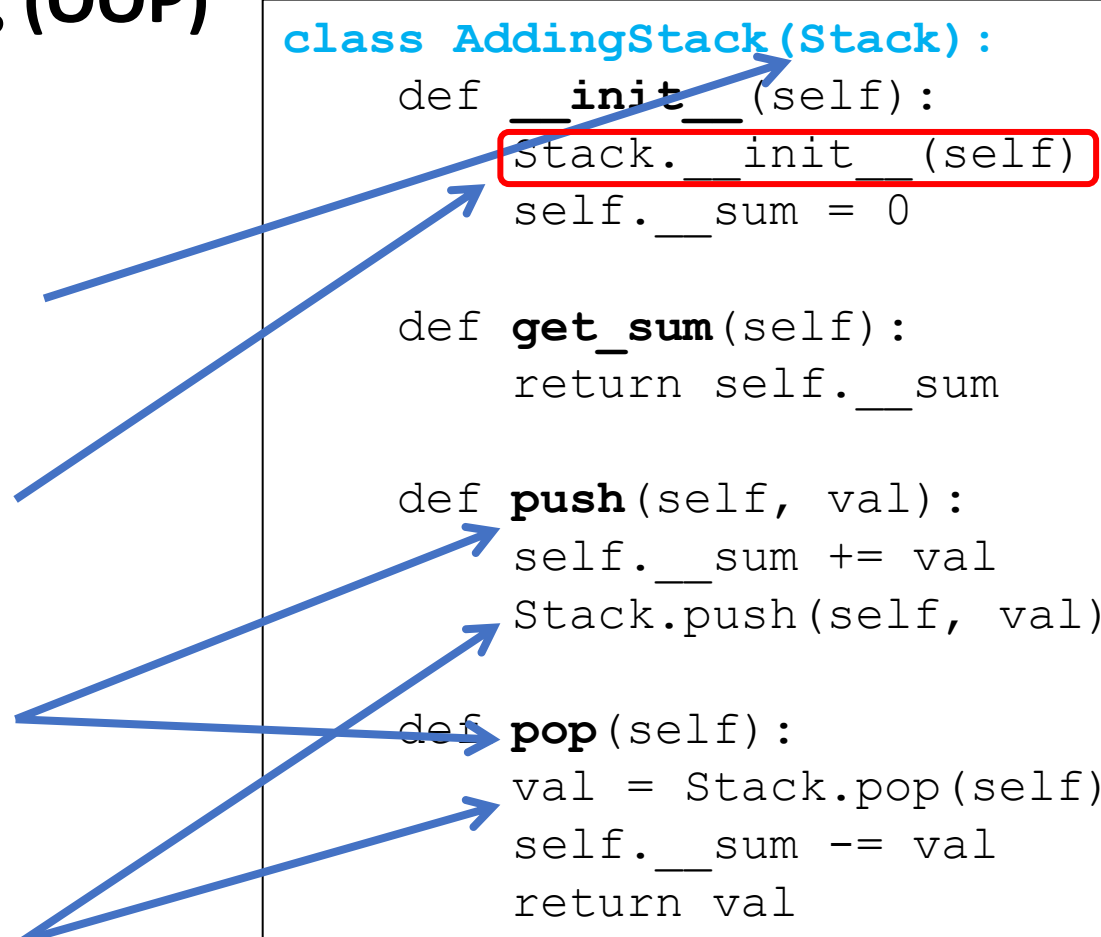
The new subclass is pointing to the class which will be used as the **superclass**. It gets all the components defined by its superclass

Contrary to many other languages, Python forces to **explicitly invoke a superclass's constructor**

Overriding - changes the functionality of the methods from the superclass

invocation of the previous implementation of the push and pop methods (available in the superclass)

```
class AddingStack(Stack):  
    def __init__(self):  
        Stack.__init__(self)  
        self.__sum = 0  
  
    def get_sum(self):  
        return self.__sum  
  
    def push(self, val):  
        self.__sum += val  
        Stack.push(self, val)  
  
    def pop(self):  
        val = Stack.pop(self)  
        self.__sum -= val  
        return val
```



Python Language Programming

• Object Oriented Programming (OOP)

```
class Stack:
    def __init__(self):
        self.__stack_list = []

    def push(self, val):
        self.__stack_list.append(val)

    def pop(self):
        val = self.__stack_list[-1]
        del self.__stack_list[-1]
        return val
```

```
class AddingStack(Stack):
    def __init__(self):
        Stack.__init__(self)
        self.__sum = 0

    def get_sum(self):
        return self.__sum

    def push(self, val):
        self.__sum += val
        Stack.push(self, val)

    def pop(self):
        val = Stack.pop(self)
        self.__sum -= val
        return val
```

```
stack_object = AddingStack()

for i in range(5):
    stack_object.push(i)
print(stack_object.get_sum())

for i in range(5):
    print(stack_object.pop())
```



10
4
3
2
1
0

Python Language Programming

- Object Oriented Programming (OOP)
 - Functions and Operators
 - `issubclass()`: if a class is a subclass of other class
 - `isinstance()`: if an object is of a certain class or not.
 - **`is`** operator: checks whether two variables refer to the same object.
 - Note: assigning a value of an object variable to another variable doesn't copy the object, but only its handle.

```
class SampleClass:
    def __init__(self, val):
        self.val = val

object_1 = SampleClass(0)
object_2 = SampleClass(2)
object_3 = object_1
object_3.val += 1

print(object_1 is object_2)
print(object_2 is object_3)
print(object_3 is object_1)
print(object_1.val, object_2.val, object_3.val)

string_1 = "Mary had a little "
string_2 = "Mary had a little lamb"
string_1 += "lamb"

print(string_1 == string_2, string_1 is string_2)
```



False

False

True

1 2 1

True False

Bibliography

- Burkov, A. (2019). The hundred-page machine learning book. Andriy Burkov.
- **Cisco. (2022). Python Essentials. Retrieved October 20, 2024, from <https://www.netacad.com/>**
- Geron, A. (2019). Hands-on machine learning with scikit-learn, keras, and tensorflow. O'Reilly.
- Ekman, M. (2022). Learning Deep Learning. Addison-Wesley.
- Mckinney, W. (2017). Python for data analysis: Data wrangling with pandas, numpy, and ipython. O'Reilly.
- Raschka, S., & Mirjalili, V. (2019). Python machine learning: Machine learning and deep learning with python, scikit-learn, and tensorflow. Packt Publishing.
- Russel, S., & Norvig, P. (2018). Artificial intelligence: A modern approach. Pearson Education Limited.